

Chapter 1:

Data Storage

Computer Science: An Overview
Eleventh Edition

by
J. Glenn Brookshear

Chapter 1: Data Storage

- We begin our study of computer science by considering how information is encoded and stored inside computers.
- Our first step is to discuss the basics of a computer's data storage devices and then to consider how information is encoded for storage in these systems.
- We will explore today's data storage systems and how such techniques as data compression and error handling are used to overcome their shortfalls.

Chapter 1: Data Storage

- 1.1 Bits and Their Storage
- 1.2 Main Memory
- 1.3 Mass Storage
- 1.4 Representing Information as Bit Patterns
- 1.5 The Binary System

Chapter 1: Data Storage (continued)

- 1.6 Storing Integers
- 1.7 Storing Fractions
- 1.8 Data Compression
- 1.9 Communications Errors

Bits and Bit Patterns

- **Bit:** Binary Digit (0 or 1)
- Bit Patterns are used to represent information.
 - Numbers
 - Text characters
 - Images
 - Sound
 - And others

Boolean Operations

- **Boolean Operation:** An operation that manipulates one or more true/false values
- Specific operations
 - AND, OR, XOR (exclusive or), NOT
- Operations that manipulate true/false values are called **Boolean operations, in honor of the mathematician George Boole** (1815–1864), who was a pioneer in the field of mathematics called logic.

Figure 1.1 The Boolean operations AND, OR, and XOR (exclusive or)

The AND operation

$$\begin{array}{r} 0 \\ \text{AND } 0 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 0 \\ \text{AND } 1 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 1 \\ \text{AND } 0 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 1 \\ \text{AND } 1 \\ \hline 1 \end{array}$$

The OR operation

$$\begin{array}{r} 0 \\ \text{OR } 0 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 0 \\ \text{OR } 1 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 1 \\ \text{OR } 0 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 1 \\ \text{OR } 1 \\ \hline 1 \end{array}$$

The XOR operation

$$\begin{array}{r} 0 \\ \text{XOR } 0 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 0 \\ \text{XOR } 1 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 1 \\ \text{XOR } 0 \\ \hline 1 \end{array}$$

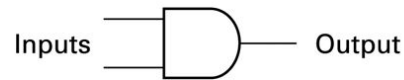
$$\begin{array}{r} 1 \\ \text{XOR } 1 \\ \hline 0 \end{array}$$

Gates

- **Gate:** A device that computes a Boolean operation
 - Often implemented as (small) electronic circuits
 - Provide the building blocks from which computers are constructed
 - VLSI (Very Large Scale Integration)

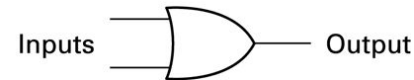
Figure 1.2 A pictorial representation of AND, OR, XOR, and NOT gates as well as their input and output values

AND



Inputs	Output
0 0	0
0 1	0
1 0	0
1 1	1

OR



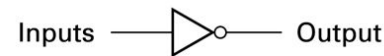
Inputs	Output
0 0	0
0 1	1
1 0	1
1 1	1

XOR



Inputs	Output
0 0	0
0 1	1
1 0	1
1 1	0

NOT



Inputs	Output
0	1
1	0

Flip-flops

- **Flip-flop:** A circuit built from gates that can store one bit.
 - One input line is used to set its stored value to 1
 - One input line is used to set its stored value to 0
 - While both input lines are 0, the most recently stored value is preserved

Flip-flops

- A FF is a circuit that produces an output value of 0 or 1, which remains constant until a pulse (a temporary change to a 1 that returns to 0) from another circuit causes it to shift to the other value (the output will flip or flop between two values under control of external stimuli).
- As long as both inputs remain 0, the output (whether 0 or 1) will not change.
- However, temporarily placing a 1 on the upper input will force the output to be 1, whereas temporarily placing a 1 on the lower input will force the output to be 0.

Figure 1.3 A simple flip-flop circuit

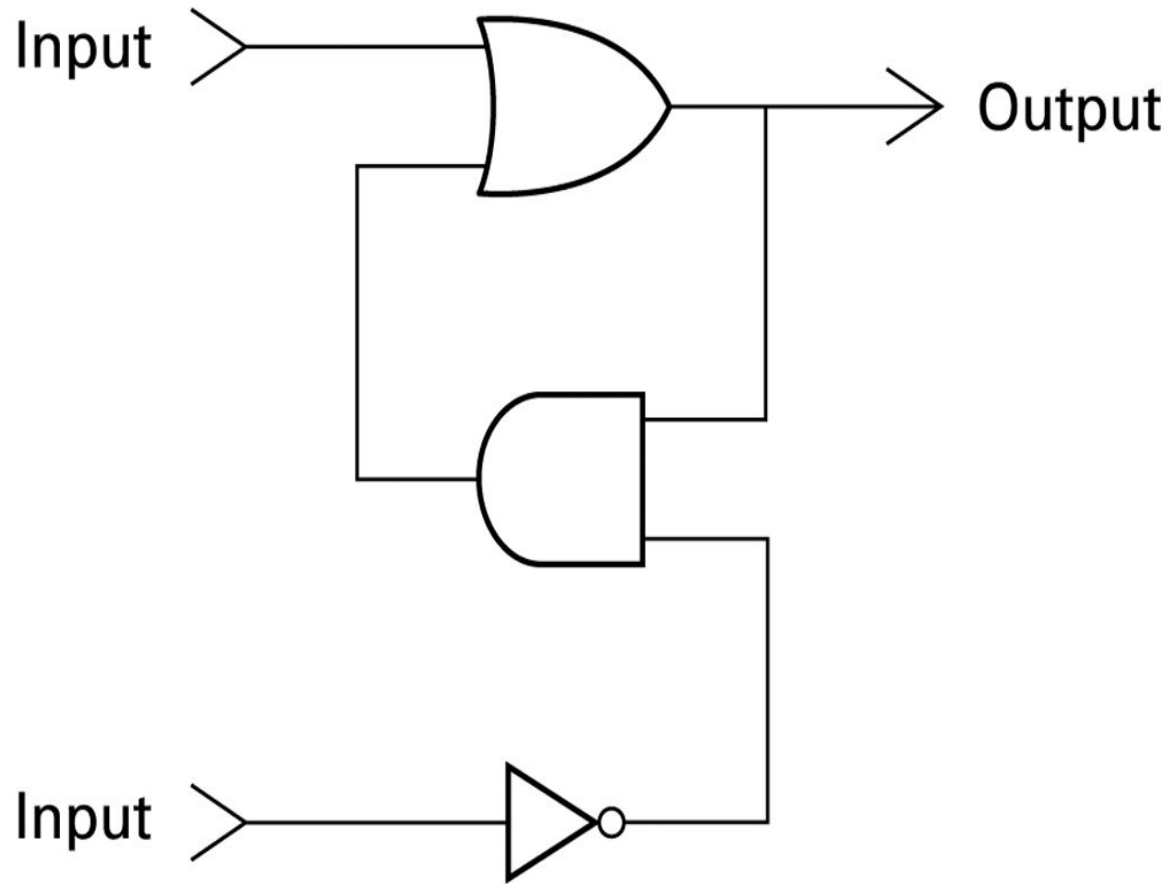


Figure 1.4 Setting the output of a flip-flop to 1

a. 1 is placed on the upper input.

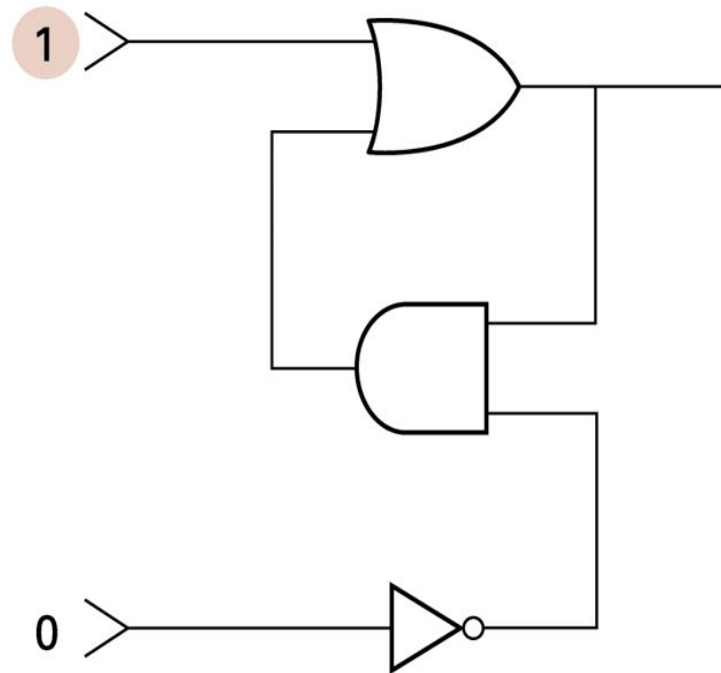


Figure 1.4 Setting the output of a flip-flop to 1 (continued)

- b.** This causes the output of the OR gate to be 1 and, in turn, the output of the AND gate to be 1.

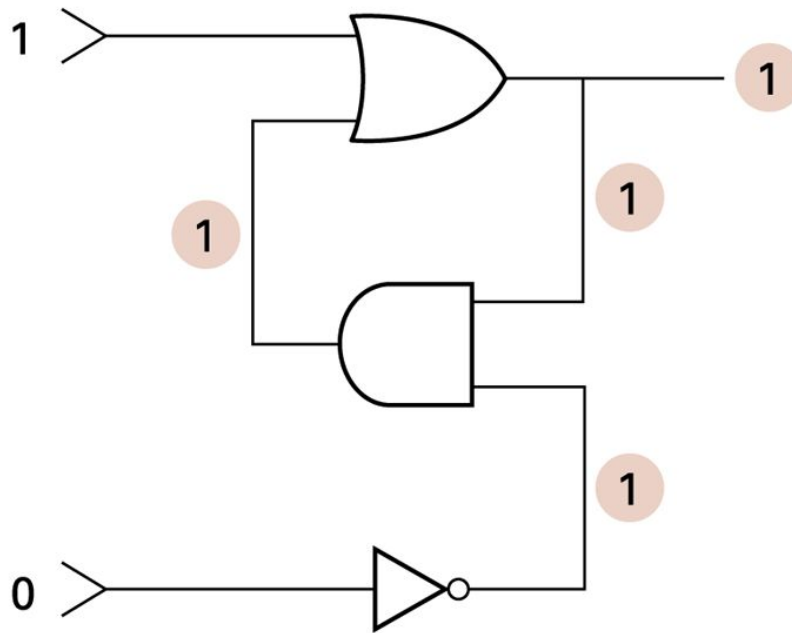
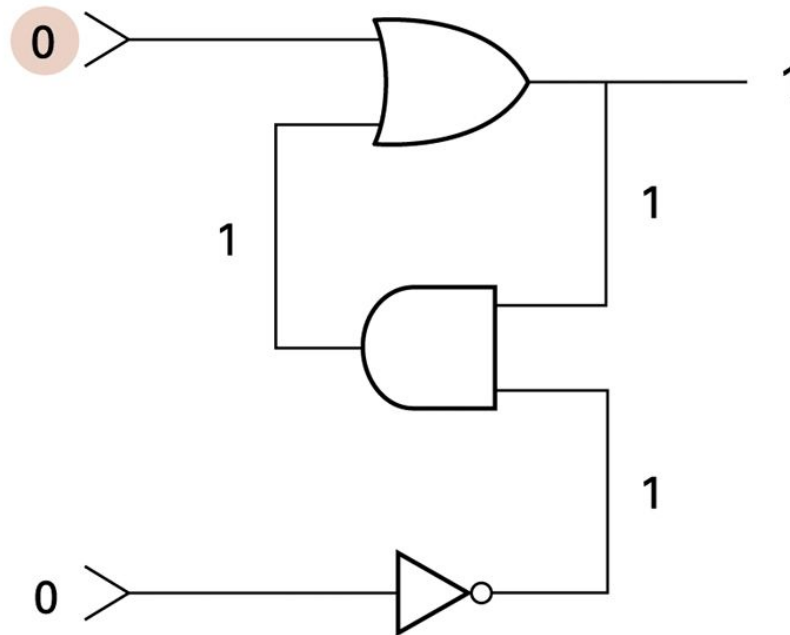


Figure 1.4 Setting the output of a flip-flop to 1 (continued)

- c. The 1 from the AND gate keeps the OR gate from changing after the upper input returns to 0.



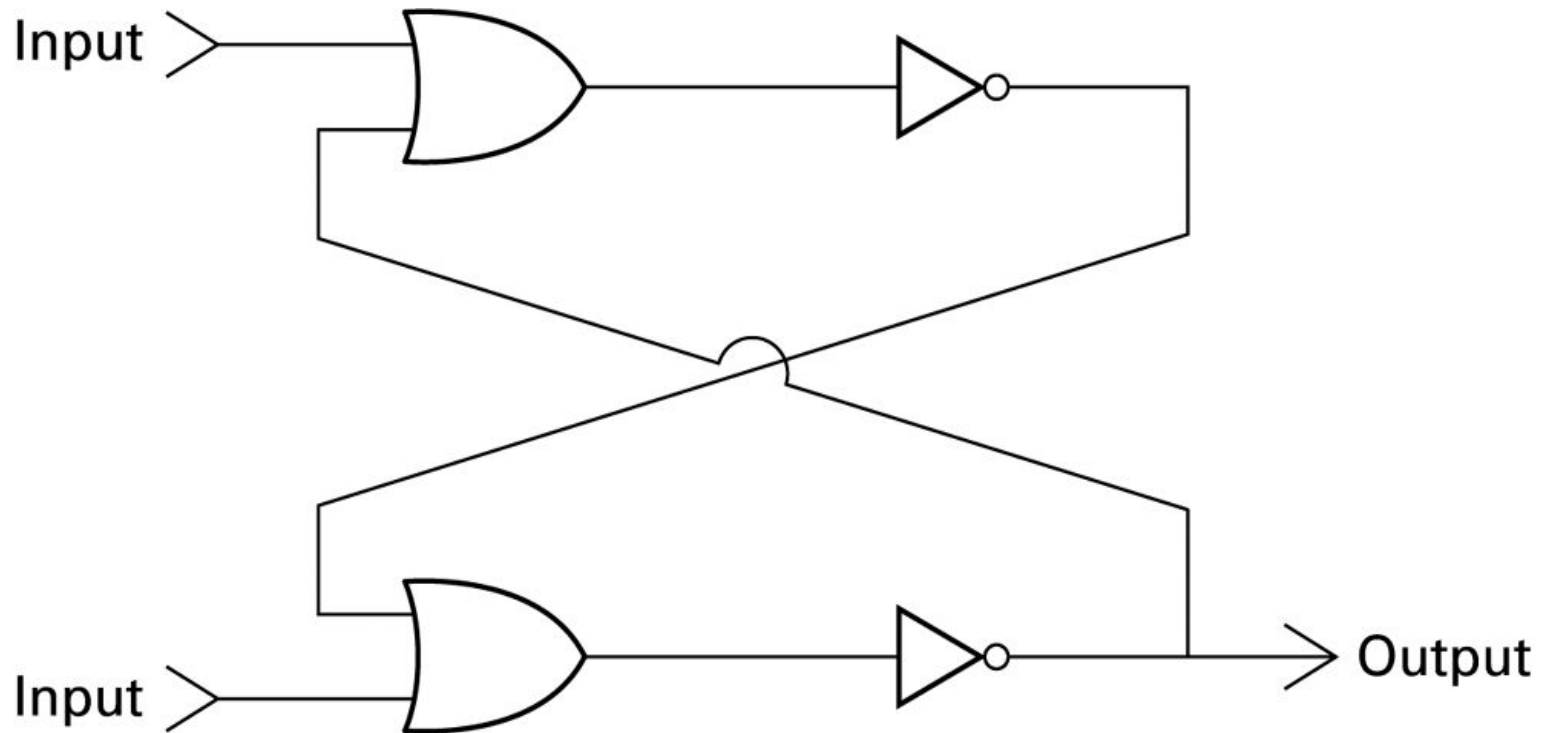
Flip-flops

- Our purpose in introducing the flip-flop circuit in Figures 1.3 and 1.4 is threefold.
- First, it demonstrates how devices can be constructed from gates, a process known as digital circuit design, which is an important topic in computer engineering.
- Indeed, the flip-flop is only one of many circuits that are basic tools in computer engineering.

Flip-flops

- Second, the concept of a flip-flop provides an example of abstraction and the use of abstract tools.
- Actually, there are other ways to build a flip-flop (Figure 1.5). Although it has a different internal structure, its external properties are the same as those of Figure 1.3.
- A computer engineer does not need to know which circuit is actually used within a flip-flop. Instead, only an understanding of the flip-flop's external properties is needed to use it as an abstract tool.

Figure 1.5 Another way of constructing a flip-flop



Flip-flops

- The third purpose for introducing the flip-flop is that it is one means of storing a bit within a modern computer (either 0 or 1). Other circuits can adjust this value by sending pulses to the flip-flop's inputs, and still other circuits can respond to the stored value by using the flip-flop's output as their inputs.
- Technology known as **very large-scale integration (VLSI)**, which allows millions of flip-flops along with their controlling circuitry to be constructed on a **chip**, is used to create miniature devices containing.

Hexadecimal Notation

- **Hexadecimal notation:** A shorthand notation for long bit patterns
 - Divides a pattern into groups of four bits each
 - Represents each group by a single symbol
- Example: 10100011 becomes A3
- In this manner, the 16-bit pattern 1010010011001000 can be reduced to the more palatable form A4C8.

Figure 1.6 The hexadecimal coding system

Bit pattern	Hexadecimal representation
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

1. What input bit patterns will cause the following circuit to produce an output of 1?



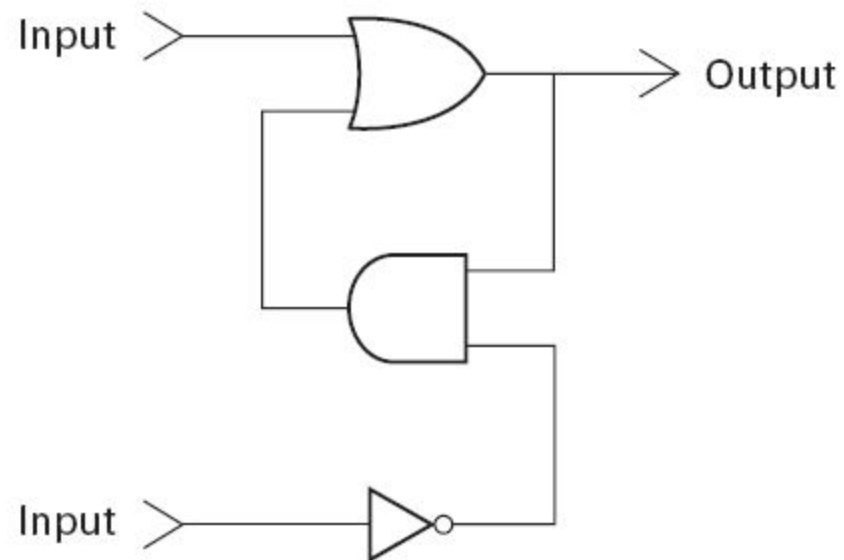
1. What input bit patterns will cause the following circuit to produce an output of 1?



1. One and only one of the upper two inputs must be 1, and the lowest input must be 1.

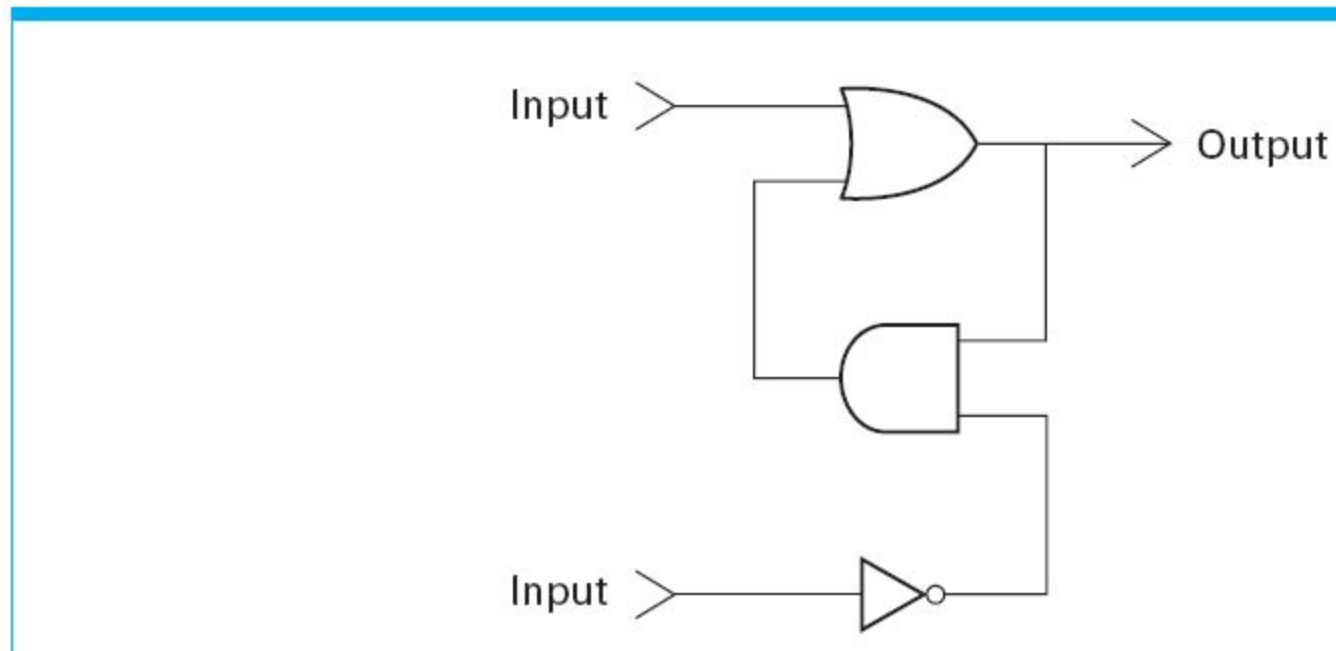
2. In the text, we claimed that placing a 1 on the lower input of the flip-flop in Figure 1.3 (while holding the upper input at 0) will force the flip-flop's output to be 0. Describe the sequence of events that occurs within the flip-flop in this case.

Figure 1.3 A simple flip-flop circuit



2. In the text, we claimed that placing a 1 on the lower input of the flip-flop in Figure 1.3 (while holding the upper input at 0) will force the flip-flop's output to be 0. Describe the sequence of events that occurs within the flip-flop in this case.

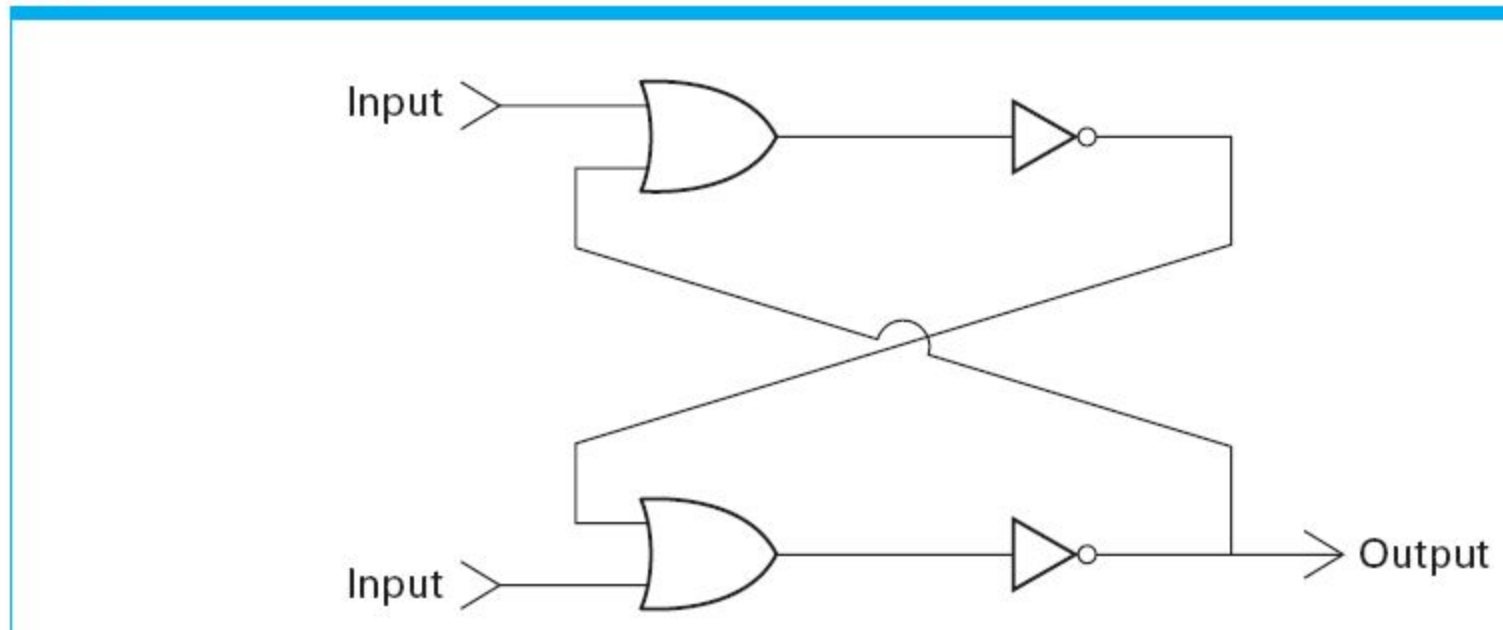
Figure 1.3 A simple flip-flop circuit



2. The 1 on the lower input is negated to 0 by the NOT gate, causing the output of the AND gate to become 0. Thus both inputs to the OR gate are 0 (remember that the upper input to the flip-flop is held at 0) so the output of the OR gate becomes 0. This means that the output of the AND gate will remain 0 after the lower input to the flip-flop returns to 0.

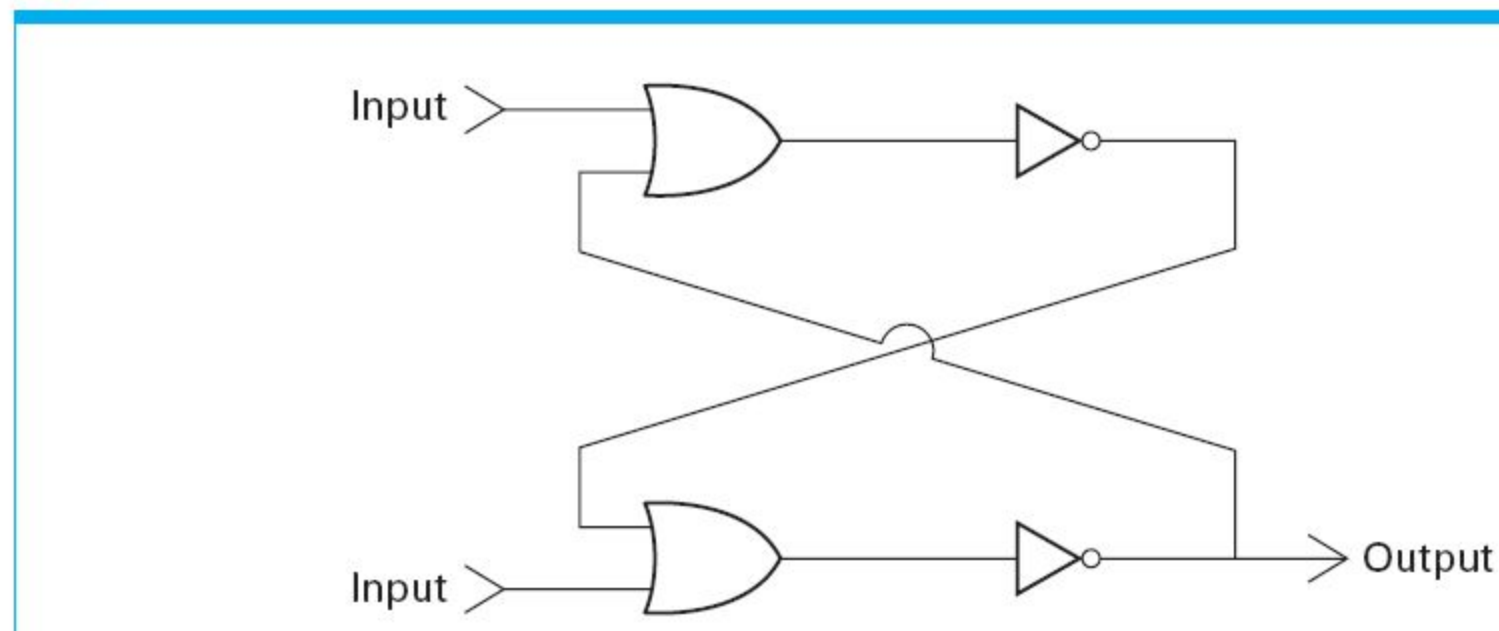
3. Assuming that both inputs to the flip-flop in Figure 1.5 are 0, describe the sequence of events that occurs when the upper input is temporarily set to 1.

Figure 1.5 Another way of constructing a flip-flop



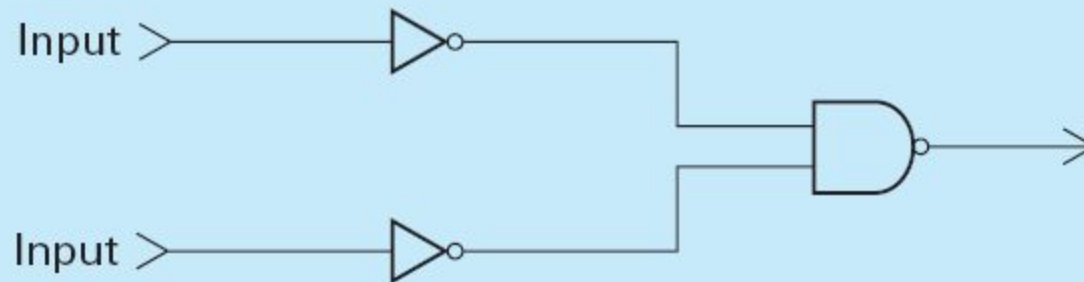
3. Assuming that both inputs to the flip-flop in Figure 1.5 are 0, describe the sequence of events that occurs when the upper input is temporarily set to 1.

Figure 1.5 Another way of constructing a flip-flop

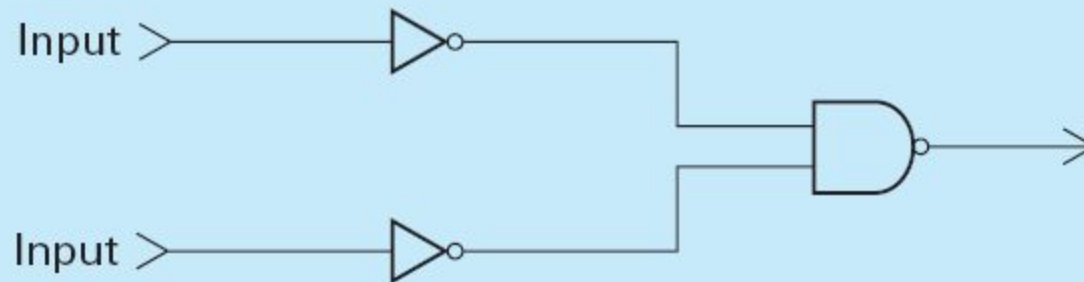


3. The output of the upper OR gate will become 1, causing the upper NOT gate to produce an output of 0. This will cause the lower OR gate to produce a 0, causing the lower NOT gate to produce a 1. This 1 is seen as the output of the flip-flop as well as being fed back to the upper OR gate, where it holds the output of that gate at 1, even after the flip-flop's input has returned to 0.

4. a. If the output of an AND gate is passed through a NOT gate, the combination computes the Boolean operation called NAND, which has an output of 0 only when both its inputs are 1. The symbol for a NAND gate is the same as an AND gate except that it has a circle at its output. The following is a circuit containing a NAND gate. What Boolean operation does the circuit compute?

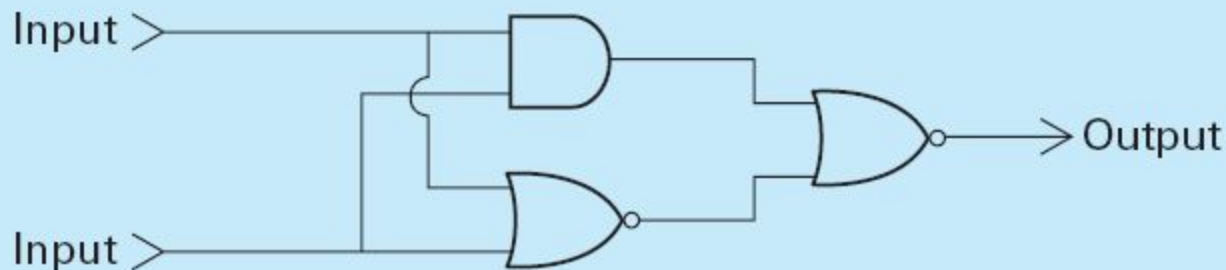


4. a. If the output of an AND gate is passed through a NOT gate, the combination computes the Boolean operation called NAND, which has an output of 0 only when both its inputs are 1. The symbol for a NAND gate is the same as an AND gate except that it has a circle at its output. The following is a circuit containing a NAND gate. What Boolean operation does the circuit compute?

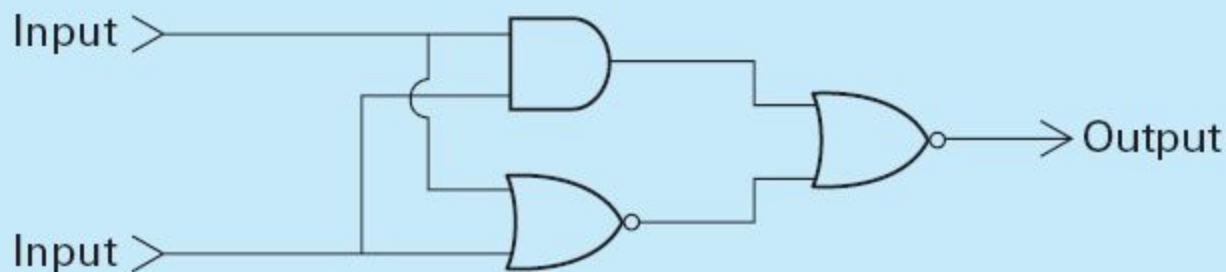


- a. The entire circuit is equivalent to a single OR gate.

- b.** If the output of an OR gate is passed through a NOT gate, the combination computes the Boolean operation called NOR that has an output of 1 only when both its inputs are 0. The symbol for a NOR gate is the same as an OR gate except that it has a circle at its output. The following is a circuit containing an AND gate and two NOR gates. What Boolean operation does the circuit compute?



- b. If the output of an OR gate is passed through a NOT gate, the combination computes the Boolean operation called NOR that has an output of 1 only when both its inputs are 0. The symbol for a NOR gate is the same as an OR gate except that it has a circle at its output. The following is a circuit containing an AND gate and two NOR gates. What Boolean operation does the circuit compute?



- b. This entire circuit is also equivalent to a single XOR gate.

5. Use hexadecimal notation to represent the following bit patterns:

a. 0110101011110010

b. 111010000101010100010111

c. 01001000

5. Use hexadecimal notation to represent the following bit patterns:

a. 0110101011110010

b. 111010000101010100010111

c. 01001000

5. a. 6AF2 b. E85517 c. 48

6. What bit patterns are represented by the following hexadecimal patterns?

a. 5FD97

b. 610A

c. ABCD

d. 0100

6. What bit patterns are represented by the following hexadecimal patterns?

a. 5FD97

b. 610A

c. ABCD

d. 0100

6. a. 01011111110110010111
b. 0110000100001010
c. 1010101111001101
d. 0000000100000000

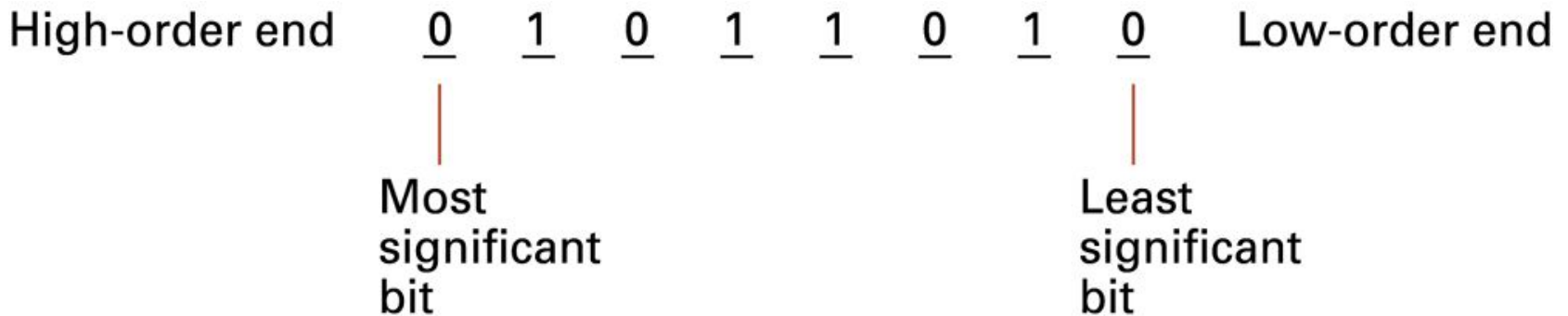
Main Memory

- For the purpose of storing data, a computer contains a large collection of circuits (such as flip-flops), each capable of storing a single bit.
- This bit reservoir is known as the machine's **main memory**.

Main Memory Cells

- **Cell:** A unit of main memory (typically 8 bits which is one **byte**)
 - **Most significant bit:** the bit at the left (high-order) end of the conceptual row of bits in a memory cell
 - **Least significant bit:** the bit at the right (low-order) end of the conceptual row of bits in a memory cell

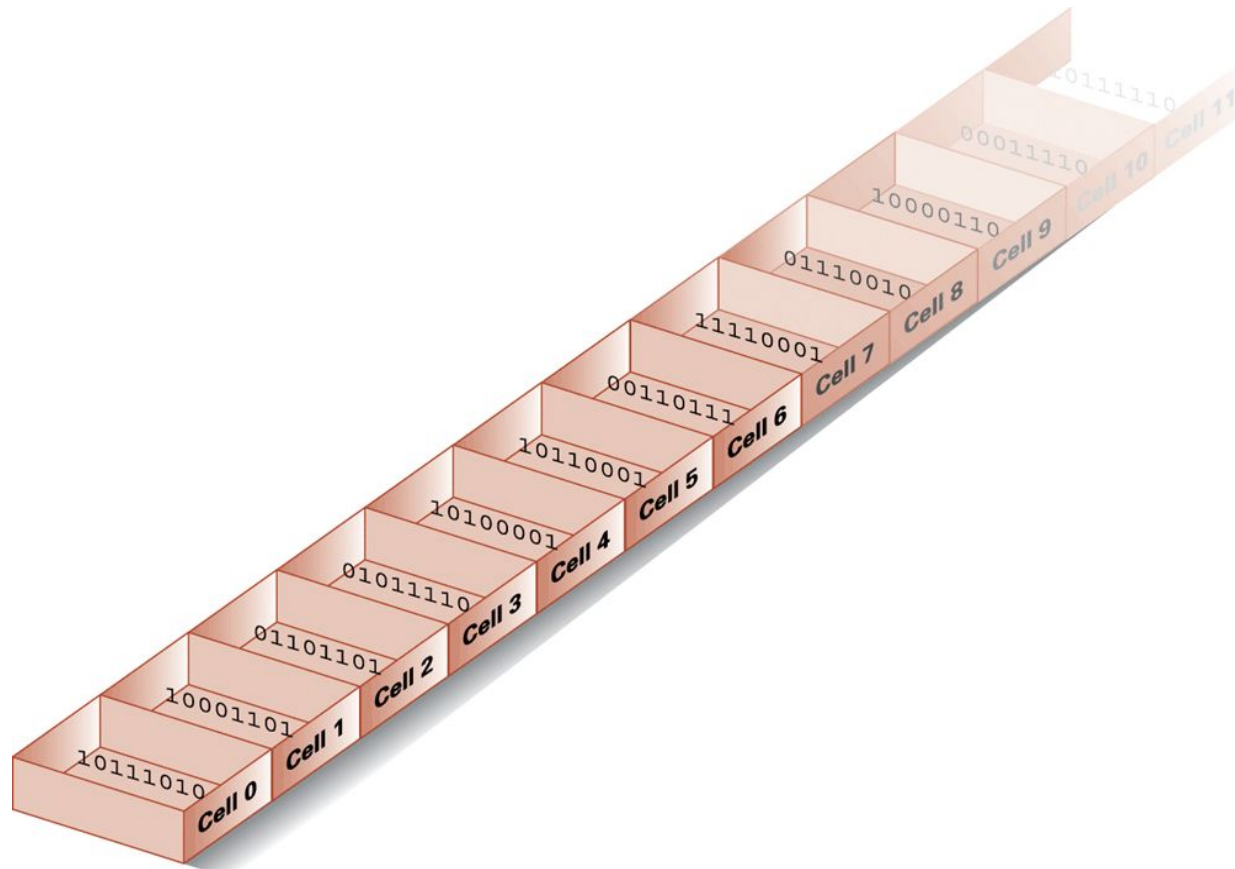
Figure 1.7 The organization of a byte-size memory cell



Main Memory Addresses

- **Address:** A “name” that uniquely identifies one cell in the computer’s main memory
 - The names are actually numbers.
 - These numbers are assigned consecutively starting at zero.
 - Numbering the cells in this manner associates an order with the memory cells.

Figure 1.8 Memory cells arranged by address



Memory Terminology

- **Random Access Memory (RAM):** Memory in which individual cells can be easily accessed in any order
- Although we have introduced flip-flops as a means of storing bits, the RAM in most modern computers is constructed using other technologies that provide greater miniaturization and faster response time. Many of these technologies store bits as tiny electric charges that dissipate quickly.

Memory Terminology

- Thus these devices require additional circuitry, known as a refresh circuit, that repeatedly replenishes the charges many times a second.
- **Dynamic Memory (DRAM):** In recognition of this volatility, computer memory constructed from such technology is often called **dynamic memory, leading to the term DRAM** (pronounced “DEE–ram”) meaning **Dynamic RAM**.

Measuring Memory Capacity

- **Kilobyte:** 2^{10} bytes = 1024 bytes
 - Example: 3 KB = 3 times 1024 bytes
- **Megabyte:** 2^{20} bytes = 1,048,576 bytes
 - Example: 3 MB = 3 times 1,048,576 bytes
- **Gigabyte:** 2^{30} bytes = 1,073,741,824 bytes
 - Example: 3 GB = 3 times 1,073,741,824 bytes

1. If the memory cell whose address is 5 contains the value 8, what is the difference between writing the value 5 into cell number 6 and moving the contents of cell number 5 into cell number 6?

1. If the memory cell whose address is 5 contains the value 8, what is the difference between writing the value 5 into cell number 6 and moving the contents of cell number 5 into cell number 6?

1. In the first case, memory cell number 6 ends up containing the value 5. In the second case, it ends up with the value 8.

2. Suppose you want to interchange the values stored in memory cells 2 and 3. What is wrong with the following sequence of steps:

Step 1. Move the contents of cell number 2 to cell number 3.

Step 2. Move the contents of cell number 3 to cell number 2.

Design a sequence of steps that correctly interchanges the contents of these cells. If needed, you may use additional cells.

2. Suppose you want to interchange the values stored in memory cells 2 and 3. What is wrong with the following sequence of steps:

Step 1. Move the contents of cell number 2 to cell number 3.

Step 2. Move the contents of cell number 3 to cell number 2.

Design a sequence of steps that correctly interchanges the contents of these cells. If needed, you may use additional cells.

2. Step 1 erases the original value in cell number 3 when the new value is written there. Consequently, Step 2 does not place the original value from cell number 3 in cell number 2. The result is that both cells end up with the value that was originally in cell number 2. A correct procedure is the following:

Step 1. Move the contents of cell number 2 to cell number 1.

Step 2. Move the contents of cell number 3 to cell number 2.

Step 3. Move the contents of cell number 1 to cell number 3.

3. How many bits would be in the memory of a computer with 4KB memory?

3. How many bits would be in the memory of a computer with 4KB memory?

3. 32768 bits

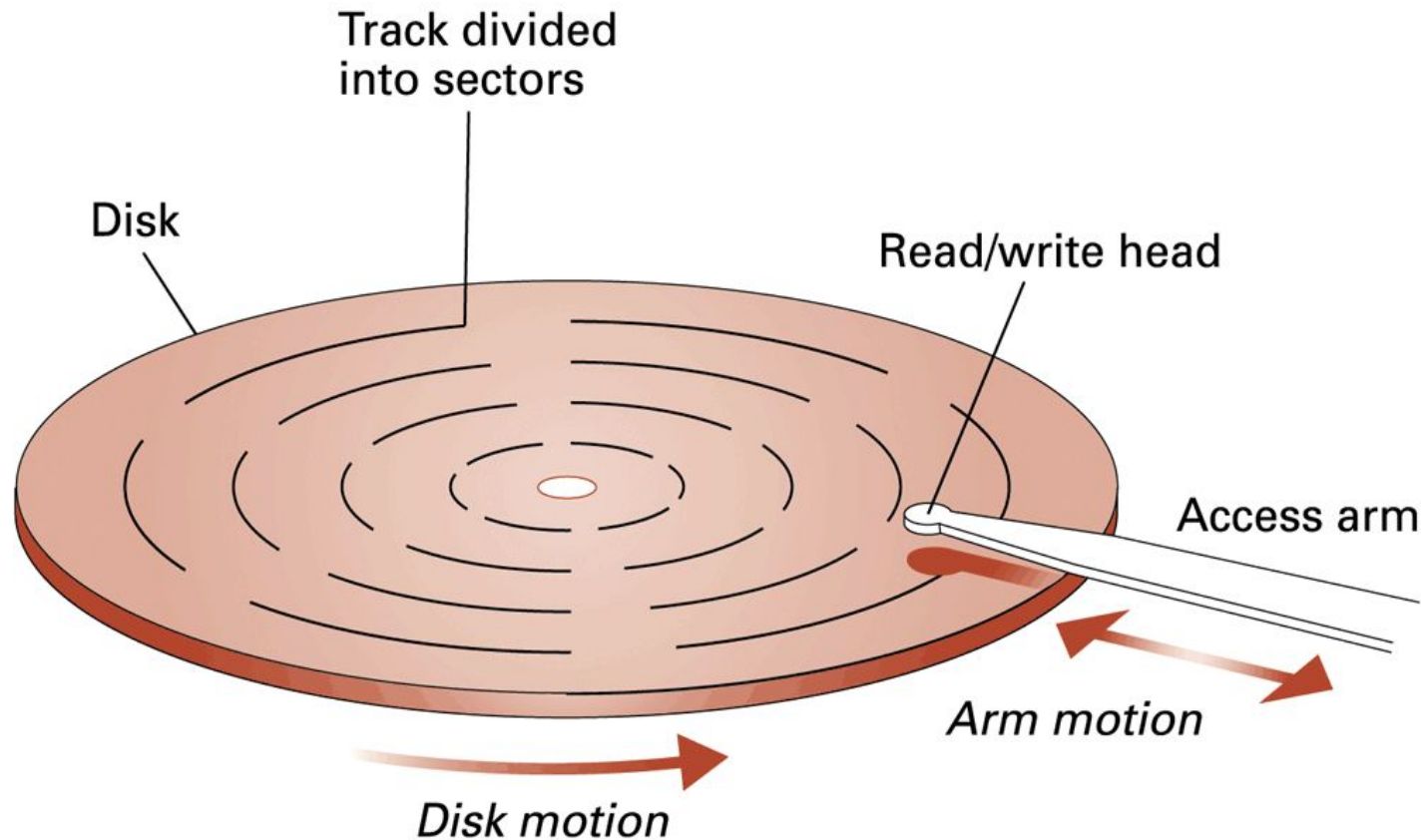
Mass Storage

- Due to the volatility and limited size of a computer's main memory, most computers have additional memory devices called **mass storage (or secondary storage) systems**, including magnetic disks, CDs, DVDs, magnetic tapes, and flash drives.
- On-line versus off-line
- Typically larger than main memory
- Typically less volatile than main memory
- Typically slower than main memory

Mass Storage Systems

- Magnetic Systems
 - Disk
 - Tape
- Optical Systems
 - CD
 - DVD
- Flash Technology
 - Flash Drives
 - Secure Digital (SD) Memory Card

Figure 1.9 A magnetic disk storage system



magnetic disk storage

- Since a track can contain more information than we would normally want to manipulate at any one time, each track is divided into small arcs called **sectors on which information is recorded as a continuous string of bits.**
- **All sectors** on a disk contain the same number of bits (typical capacities are in the range of 512 bytes to a few KB), and in the simplest disk storage systems each track contains the same number of sectors.

magnetic disk storage

- The location of tracks and sectors is not a permanent part of a disk's physical structure. Instead, they are marked magnetically through a process called **formatting (or initializing) the disk**.
- **This process is usually performed by the disk's manufacturer, resulting in what are known as formatted disks.**
- Thus, if the format information on a disk is damaged, the disk can be reformatted, although this process destroys all the information that **was previously recorded on the disk.**

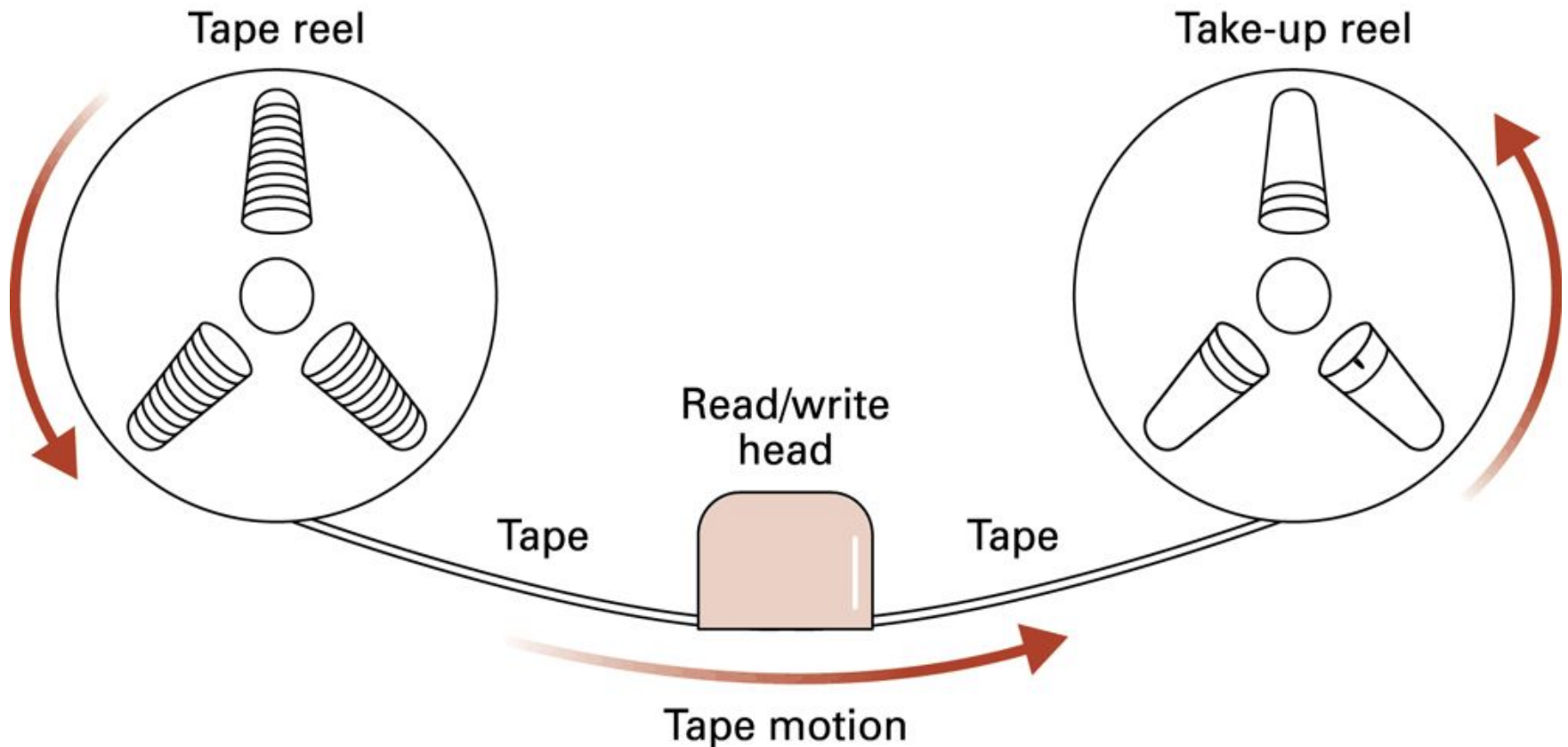
magnetic disk storage

- The capacity of a disk storage system depends on the number of platters used and the density in which the tracks and sectors are placed.
- Lower-capacity systems may consist of a single platter. High-capacity disk systems, capable of holding many gigabytes, or even terabytes, consist of perhaps three to six platters mounted on a common spindle.
- Furthermore, data may be stored on both the upper and lower surfaces of each platter.

magnetic disk storage

- evaluate a disk system's performance:
- (1) **seek time** (the time required to move the read/write heads from one track to another);
- (2) **rotation delay or latency time** (half the time required for the disk to make a complete rotation, which is the average amount of time required for the desired data to rotate around to the read/write head once the head has been positioned over the desired track);
- (3) **access time** (the sum of seek time and rotation delay); and
- (4) **transfer rate** (the rate at which data can be transferred to or from the disk).

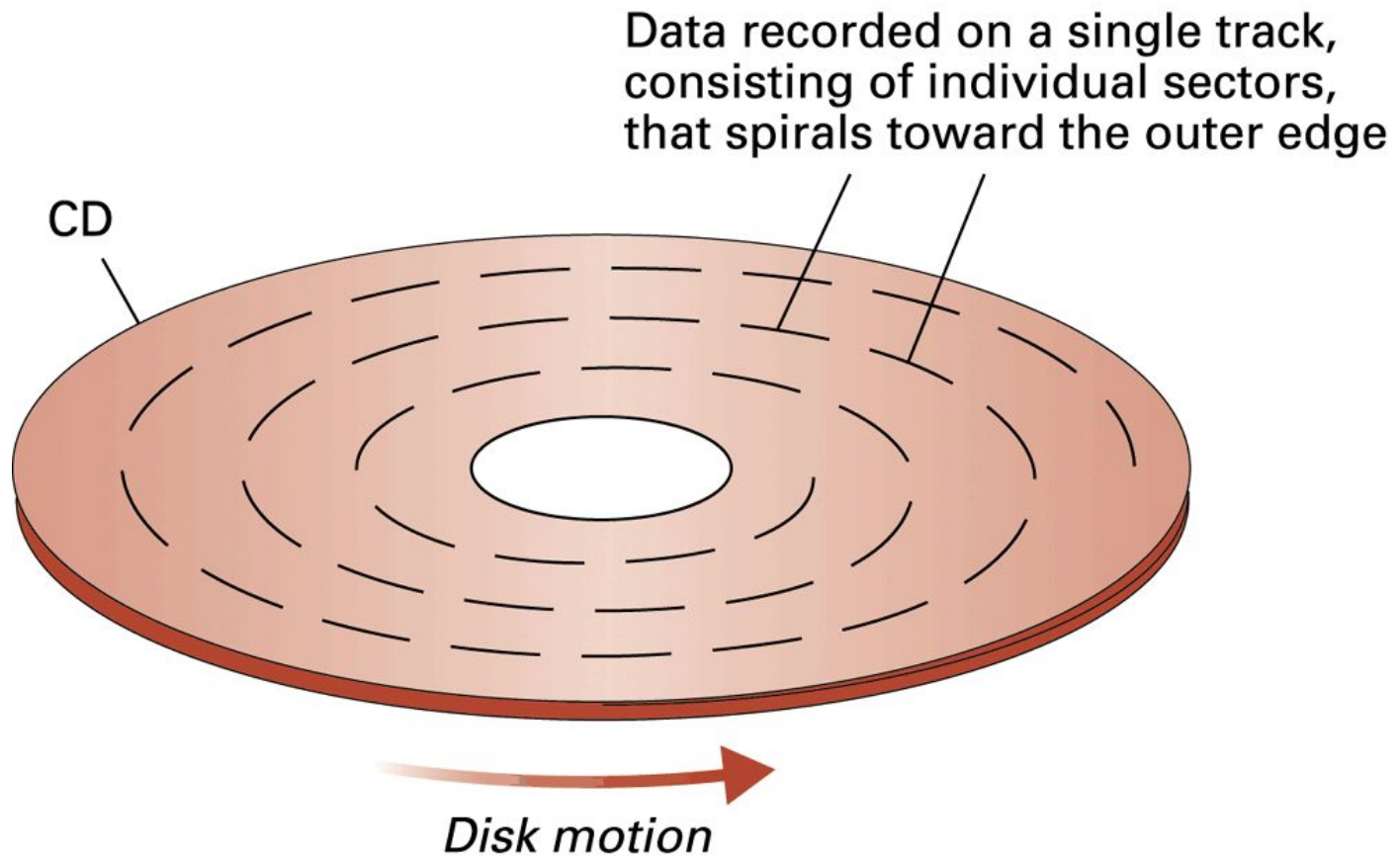
Figure 1.10 Magnetic tape storage



Optical Systems

- Another class of mass storage systems applies optical technology. An example is the **compact disk (CD)**
- These disks are 12 centimeters (approximately 5 inches) in diameter and consist of reflective material covered with a clear protective coating.
- Information is recorded on them by creating variations in their reflective surfaces. This information can then be retrieved by means of a laser beam that detects irregularities on the reflective surface of the CD as it spins.
- This track is divided into units called sectors, each with its own identifying markings and a capacity of 2KB of data, which equates to $\frac{1}{75}$ of a second of music in the case of audio recordings.

Figure 1.11 CD storage



Optical Systems

- Traditional CDs have capacities in the range of 600 to 700MB.
- However, **DVDs (Digital Versatile Disks)**, which are constructed from multiple, semitransparent layers that serve as distinct surfaces when viewed by a precisely focused laser, provide storage capacities of several GB.
- Finally, Blu-ray technology, which uses a laser in the blue-violet spectrum of light (instead of red), is able to focus its laser beam with very fine precision.
- **BDs (Blu-ray Disks)** provides over five times the capacity of a DVD. This seemingly vast amount of storage is needed to meet the demands of high

Flash Drives

- A common property of mass storage systems based on magnetic or optic technology is that physical motion, such as spinning disks, moving read/write heads, and aiming laser beams, is required to store and retrieve data. This means that data storage and retrieval is slow compared to the speed of electronic circuitry.
- **Flash memory** technology has the potential of alleviating this drawback.
- In a flash memory system, bits are stored by sending electronic signals directly to the storage medium where altering the characteristics of small electronic circuits.

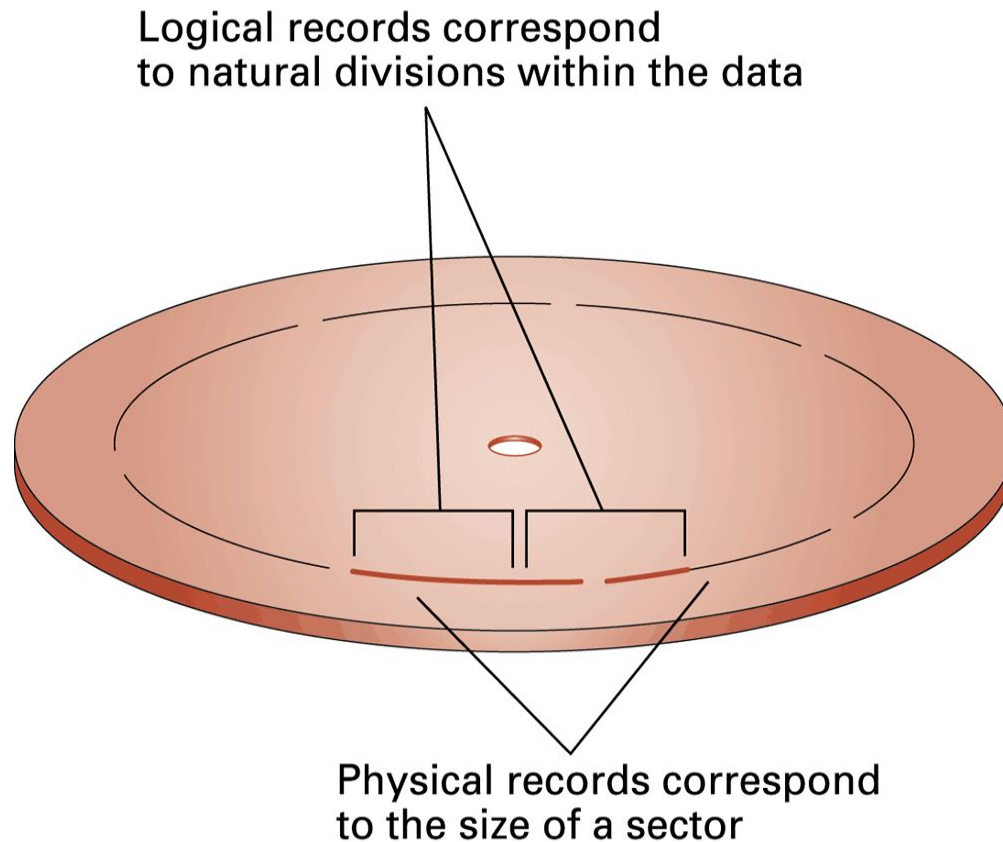
Files

- **File:** A unit of data stored in mass storage system
- Mass storage devices dictate that files be stored and retrieved in smaller, multiple byte units.
- For example, a file stored on a magnetic disk must be manipulated by sectors, each of which is a fixed predetermined size.

Files

- **Buffer:** A memory area used for the temporary storage of data (usually as a step in transferring the data)
- For example, modern printers contain memory circuitry of their own, a large part of which is used as a buffer for holding portions of a document that have been received by the printer but not yet printed.

Figure 1.12 Logical records versus physical records on a disk



Representing Text

- **Each character (letter, punctuation, etc.) is assigned a unique bit pattern.**
 - **ASCII(American Standard Code for Information Interchange):** Uses patterns of 7-bits to represent most symbols used in written English text
 - **ISO(International Organization for Standardization)** developed a number of 8 bit extensions to ASCII, each designed to accommodate a major language group
 - **Unicode:** Uses patterns of 16-bits to represent the major symbols used in languages world wide

ASCII

Symbol	ASCII	Hex	Symbol	ASCII	Hex	Symbol	ASCII	Hex
line feed	00001010	0A	>	00111110	3E	^	01011110	5E
carriage return	00001011	0B	?	00111111	3F	~	01011111	5F
space	00100000	20	@	01000000	40	`	01100000	60
!	00100001	21	A	01000001	41	a	01100001	61
"	00100010	22	B	01000010	42	b	01100010	62
#	00100011	23	C	01000011	43	c	01100011	63
\$	00100100	24	D	01000100	44	d	01100100	64
%	00100101	25	E	01000101	45	e	01100101	65
&	00100110	26	F	01000110	46	f	01100110	66
'	00100111	27	G	01000111	47	g	01100111	67
(	00101000	28	H	01001000	48	h	01101000	68
)	00101001	29	I	01001001	49	i	01101001	69
*	00101010	2A	J	01001010	4A	j	01101010	6A
+	00101011	2B	K	01001011	4B	k	01101011	6B
,	00101100	2C	L	01001100	4C	l	01101100	6C
-	00101101	2D	M	01001101	4D	m	01101101	6D
.	00101110	2E	N	01001110	4E	n	01101110	6E
/	00111111	2F	O	01001111	4F	o	01101111	6F
0	00110000	30	P	01010000	50	p	01110000	70
1	00110001	31	Q	01010001	51	q	01110001	71
2	00110010	32	R	01010010	52	r	01110010	72
3	00110011	33	S	01010011	53	s	01110011	73
4	00110100	34	T	01010100	54	t	01110100	74
5	00110101	35	U	01010101	55	u	01110101	75
6	00110110	36	V	01010110	56	v	01110110	76
7	00110111	37	W	01010111	57	w	01110111	77
8	00111000	38	X	01011000	58	x	01111000	78
9	00111001	39	Y	01011001	59	y	01111001	79
:	00111010	3A	Z	01011010	5A	z	01111010	7A
;	00111011	3B	[	01011011	5B	{	01111011	7B
<	00111100	3C	\	01011100	5C		01111100	7C
=	00111101	3D	]	01011101	5D	}	01111101	7D

Figure 1.13 The message “Hello.” in ASCII

01001000	01100101	01101100	01101100	01101111	00101110
H	e	l	l	o	.

Representing Numeric Values

- Binary notation: Uses bits to represent a number in base two
- Limitations of computer representations of numeric values
 - Overflow: occurs when a value is too big to be represented
 - Truncation: occurs when a value cannot be represented accurately

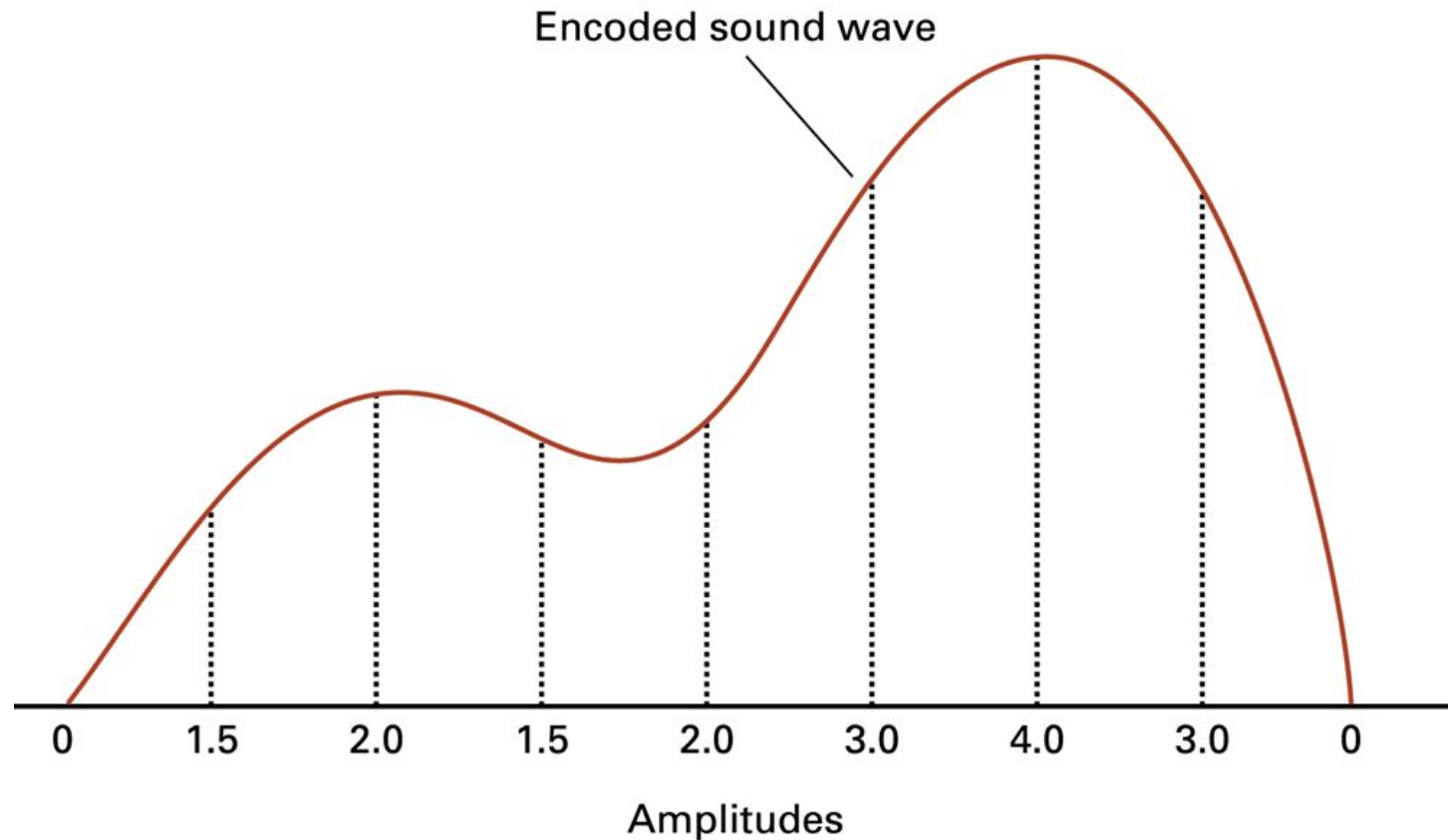
Representing Images

- Bit map techniques
 - Pixel: short for “picture element”
 - RGB
 - Luminance and chrominance
- Vector techniques
 - Scalable
 - TrueType and PostScript

Representing Sound

- Sampling techniques
 - Used for high quality recordings
 - Records actual audio
- The most generic method of encoding audio information for computer storage and manipulation is to sample the amplitude of the sound wave at regular intervals and record the series of values obtained.
- (Figure 1.14). This technique,
- using a sample rate of 8000 samples per

Figure 1.14 The sound wave represented by the sequence 0, 1.5, 2.0, 1.5, 2.0, 3.0, 4.0, 3.0, 0



a sound wave that rises in amplitude, falls briefly, rises to a higher level, and then drops back to 0.

Representing Sound

- This technique, using a sample rate of 8000 samples per second, has been used for years in long-distance voice telephone communication.
- The voice at one end of the communication is encoded as numeric values representing the amplitude of the voice every eight-thousandth of a second.
- These numeric values are then transmitted over the communication line to the receiving end, where they are used to reproduce the sound of the voice.

1. Here is a message encoded in ASCII using 8 bits per symbol. What does it say? (*See Appendix A*)

```
01000011 01101111 01101101 01110000 01110101 01110100
01100101 01110010 00100000 01010011 01100011 01101001
01100101 01101110 01100011 01100101
```

1. Here is a message encoded in ASCII using 8 bits per symbol. What does it say? (*See Appendix A*)

```
01000011 01101111 01101101 01110000 01110101 01110100  
01100101 01110010 00100000 01010011 01100011 01101001  
01100101 01101110 01100011 01100101
```

1. Computer Science

2. In the ASCII code, what is the relationship between the codes for an uppercase letter and the same letter in lowercase? (*See Appendix A.*)

- 2. In the ASCII code, what is the relationship between the codes for an uppercase letter and the same letter in lowercase? (*See Appendix A.*)
- 2. The two patterns are the same, except that the sixth bit from the low-order end is always 0 for uppercase and 1 for lowercase.

5. Convert each of the following binary representations to its equivalent base ten form:

a. 0101

b. 1001

c. 1011

d. 0110

e. 10000

f. 10010

5. Convert each of the following binary representations to its equivalent base ten form:

a. 0101

b. 1001

c. 1011

d. 0110

e. 10000

f. 10010

5. Convert each of the following binary representations to its equivalent base ten form:

a. 0101

b. 1001

c. 1011

d. 0110

e. 10000

f. 10010

5. a. 5

b. 9

c. 11

d. 6

e. 16

f. 18

6. Convert each of the following base ten representations to its equivalent binary form:

a. 6

b. 13

c. 11

d. 18

e. 27

f. 4

6. Convert each of the following base ten representations to its equivalent binary form:

a. 6

b. 13

c. 11

d. 18

e. 27

f. 4

6. a. 110 b. 1101 c. 1011 d. 10010 e. 11011 f. 100

7. What is the largest numeric value that could be represented with three bytes if each digit were encoded using one ASCII pattern per byte? What if binary notation were used?

7. What is the largest numeric value that could be represented with three bytes if each digit were encoded using one ASCII pattern per byte? What if binary notation were used?

7. In 24 bits, we can store three symbols using ASCII. Thus we can store values as large as 999. However, if we use the bits as binary digits, we can store values up to 16,777,215.

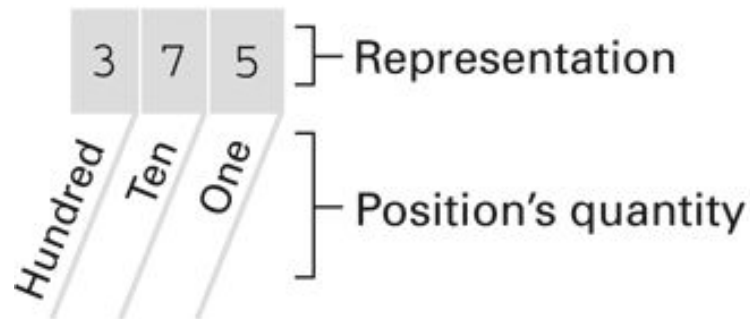
The Binary System

The traditional decimal system is based on powers of ten.

The Binary system is based on powers of two.

Figure 1.15 The base ten and binary systems

a. Base ten system



b. Base two system

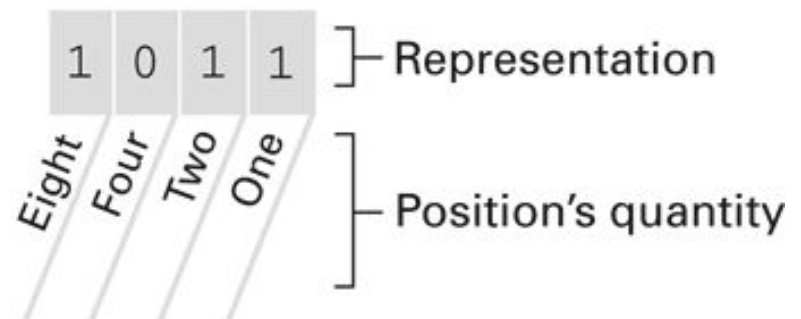


Figure 1.16 Decoding the binary representation 100101

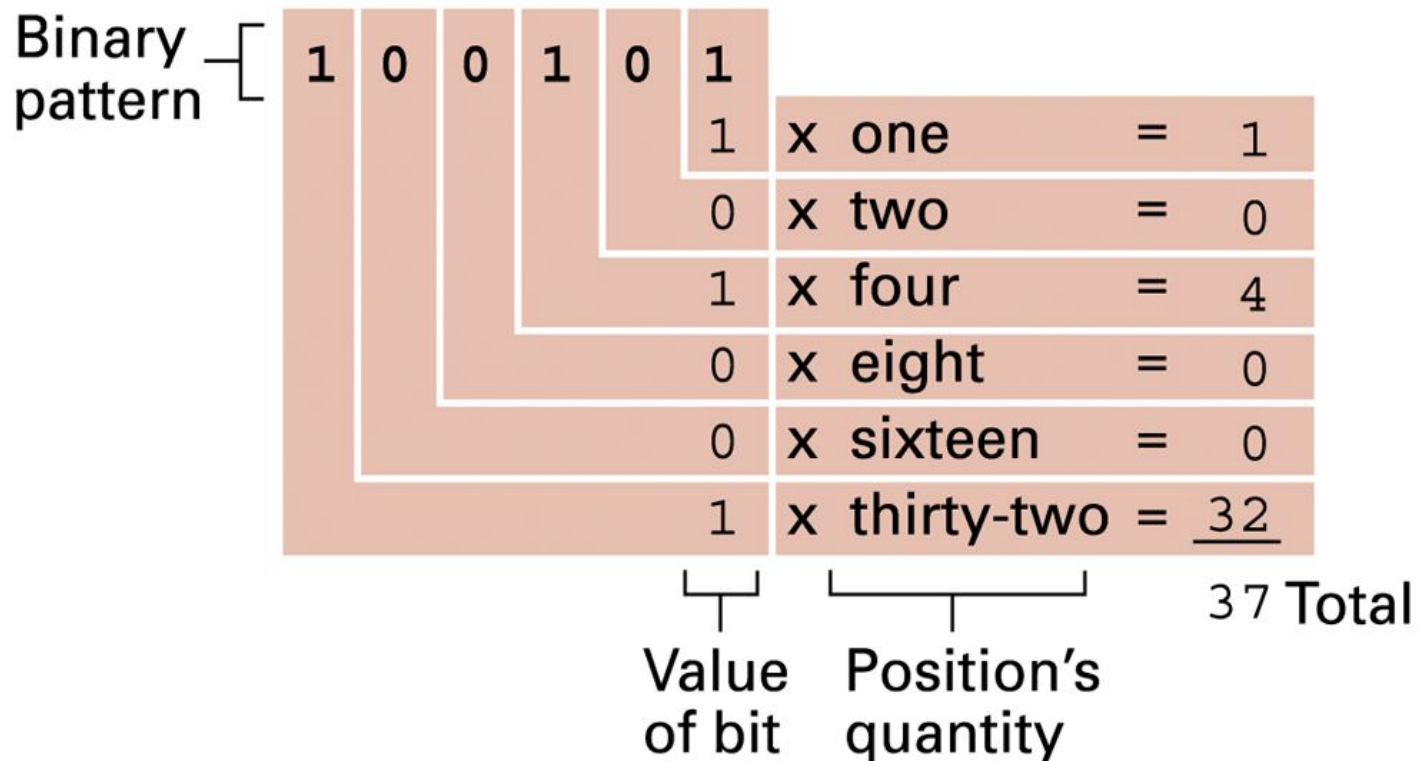


Figure 1.17 An algorithm for finding the binary representation of a positive integer

- Step 1.** Divide the value by two and record the remainder.
- Step 2.** As long as the quotient obtained is not zero, continue to divide the newest quotient by two and record the remainder.
- Step 3.** Now that a quotient of zero has been obtained, the binary representation of the original value consists of the remainders listed from right to left in the order they were recorded.

Figure 1.18 Applying the algorithm in Figure 1.15 to obtain the binary representation of thirteen

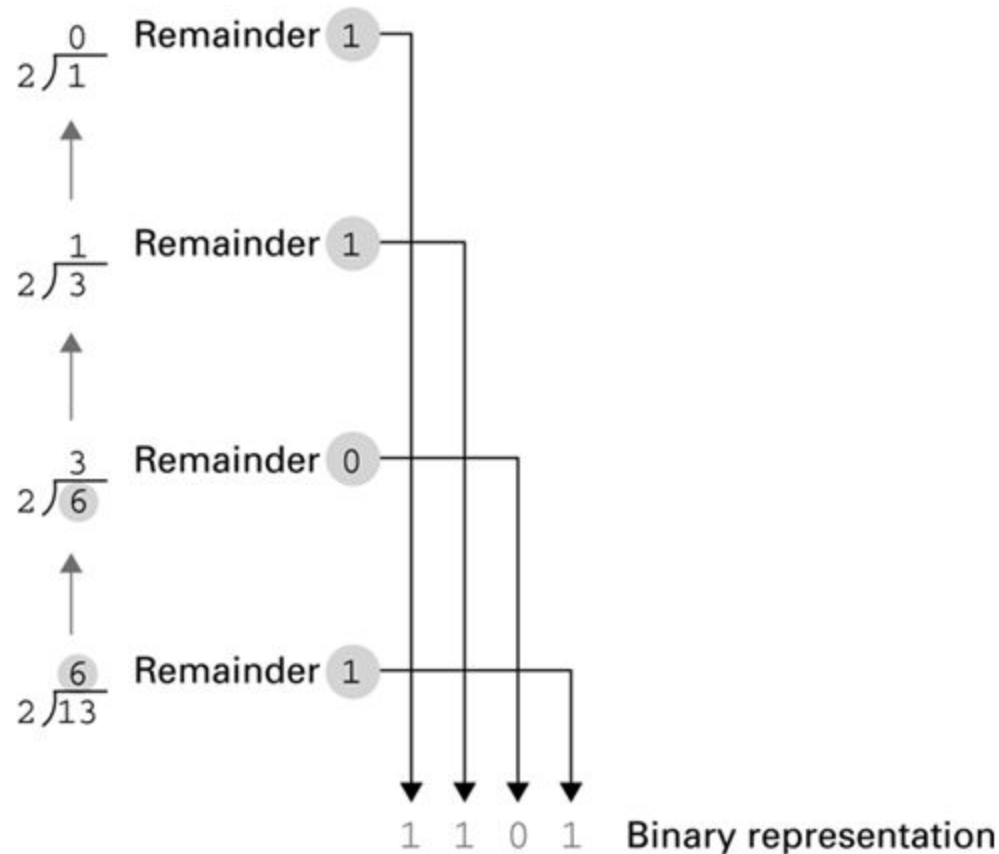


Figure 1.19 The binary addition facts

$$\begin{array}{r} 0 \\ + 0 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 1 \\ + 0 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 0 \\ + 1 \\ \hline 1 \end{array}$$

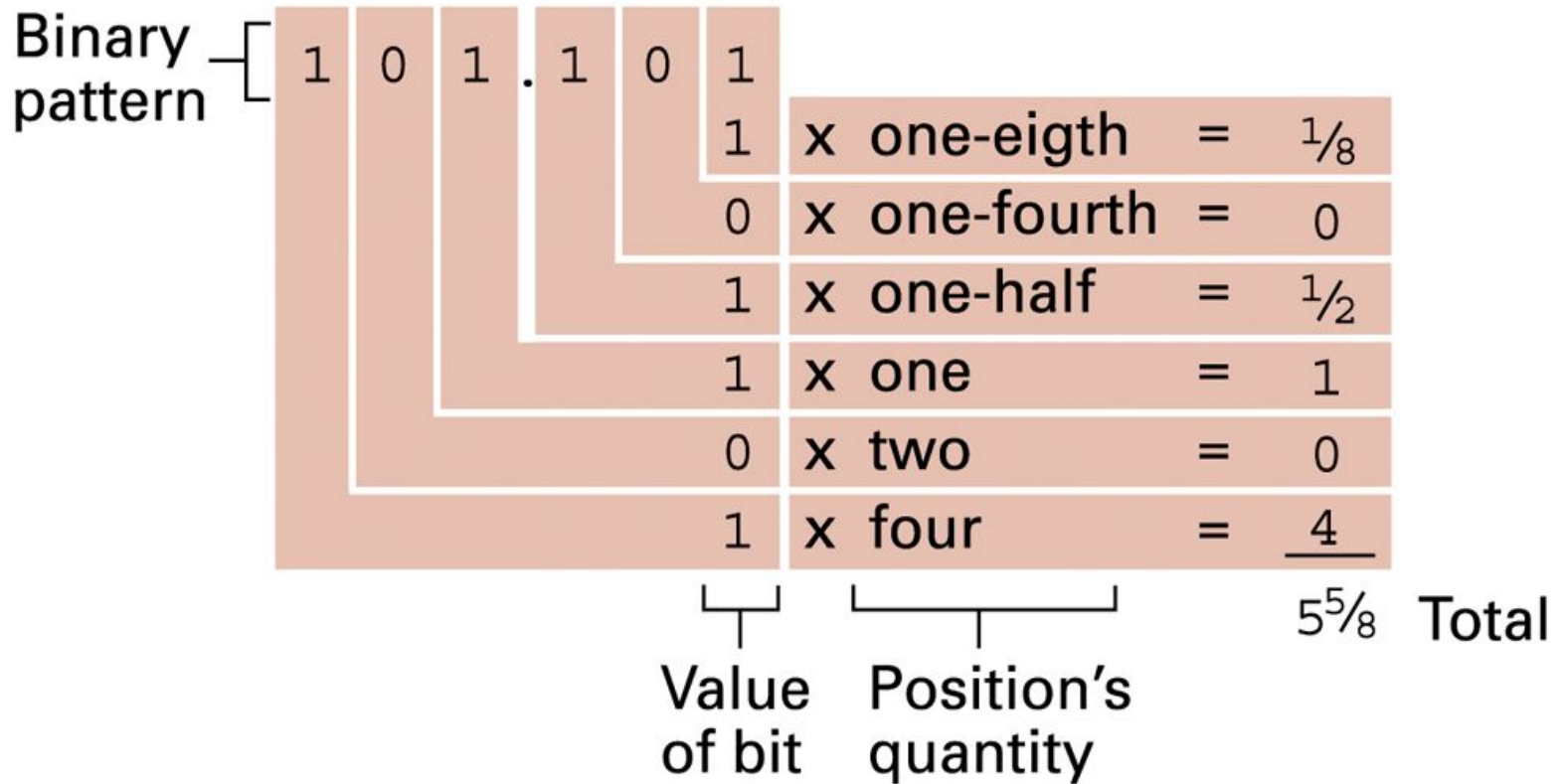
$$\begin{array}{r} 1 \\ + 1 \\ \hline 10 \end{array}$$

Binary Addition

$$\begin{array}{r} 58 \\ + 27 \\ \hline \end{array}$$
$$\begin{array}{r} 1 \\ 58 \\ + 27 \\ \hline 5 \end{array}$$
$$\begin{array}{r} 58 \\ + 27 \\ \hline 85 \end{array}$$

$$\begin{array}{r} 111010 \\ + 11011 \\ \hline \end{array}$$
$$\begin{array}{r} 1 \\ 111010 \\ + 11011 \\ \hline 01 \end{array}$$
$$\begin{array}{r} 1 \\ 111010 \\ + 11011 \\ \hline 0101 \end{array}$$
$$\begin{array}{r} 1 \\ 111010 \\ + 11011 \\ \hline 010101 \end{array}$$
$$\begin{array}{r} 111010 \\ + 11011 \\ \hline 1010101 \end{array}$$

Figure 1.20 Decoding the binary representation 101.101



1. Convert each of the following binary representations to its equivalent base ten form:
 - a. 101010 b. 100001 c. 10111 d. 0110 e. 11111
2. Convert each of the following base ten representations to its equivalent binary form:
 - a. 32 b. 64 c. 96 d. 15 e. 27
3. Convert each of the following binary representations to its equivalent base ten form:
 - a. 11.01 b. 101.111 c. 10.1 d. 110.011 e. 0.101
4. Express the following values in binary notation:
 - a. $4\frac{1}{2}$ b. $2\frac{3}{4}$ c. $1\frac{1}{8}$ d. $\frac{5}{16}$ e. $5\frac{5}{8}$
5. Perform the following additions in binary notation:

a. $\begin{array}{r} 11011 \\ +1100 \\ \hline \end{array}$	b. $\begin{array}{r} 1010.001 \\ + 1.101 \\ \hline \end{array}$	c. $\begin{array}{r} 11111 \\ + 0001 \\ \hline \end{array}$	d. $\begin{array}{r} 111.11 \\ + 00.01 \\ \hline \end{array}$
---	--	--	--

1. Convert each of the following binary representations to its equivalent base ten form:

- a. 101010 b. 100001 c. 10111 d. 0110 e. 11111

2. Convert each of the following base ten representations to its equivalent binary form:

- a. 32 b. 64 c. 96 d. 15 e. 27

3. Convert each of the following binary representations to its equivalent base ten form:

- a. 11.01 b. 101.111 c. 10.1 d. 110.011 e. 0.101

4. Express the following values in binary notation:

- a. $4\frac{1}{2}$ b. $2\frac{3}{4}$ c. $1\frac{1}{8}$ d. $\frac{5}{16}$ e. $5\frac{5}{8}$

5. Perform the following additions in binary notation:

- | | | | |
|-------------------|-------------------|-------------------|-------------------|
| a. 11011 | b. 1010.001 | c. 11111 | d. 111.11 |
| +1100 | + 1.101 | + 0001 | + 00.01 |
| <u> </u> | <u> </u> | <u> </u> | <u> </u> |

1. a. 42 b. 33 c. 23 d. 6 e. 31

1. Convert each of the following binary representations to its equivalent base ten form:

- a. 101010 b. 100001 c. 10111 d. 0110 e. 11111

2. Convert each of the following base ten representations to its equivalent binary form:

- a. 32 b. 64 c. 96 d. 15 e. 27

3. Convert each of the following binary representations to its equivalent base ten form:

- a. 11.01 b. 101.111 c. 10.1 d. 110.011 e. 0.101

4. Express the following values in binary notation:

- a. $4\frac{1}{2}$ b. $2\frac{3}{4}$ c. $1\frac{1}{8}$ d. $\frac{5}{16}$ e. $5\frac{5}{8}$

5. Perform the following additions in binary notation:

- | | | | |
|---------------|---------------|---------------|---------------|
| a. 11011 | b. 1010.001 | c. 11111 | d. 111.11 |
| +1100 | + 1.101 | + 0001 | + 00.01 |
| <u> </u> | <u> </u> | <u> </u> | <u> </u> |

2. a. 100000 b. 1000000 c. 1100000 d. 1111 e. 11011

1. Convert each of the following binary representations to its equivalent base ten form:
 - a. 101010 b. 100001 c. 10111 d. 0110 e. 11111
2. Convert each of the following base ten representations to its equivalent binary form:
 - a. 32 b. 64 c. 96 d. 15 e. 27
3. Convert each of the following binary representations to its equivalent base ten form:
 - a. 11.01 b. 101.111 c. 10.1 d. 110.011 e. 0.101
4. Express the following values in binary notation:
 - a. $4\frac{1}{2}$ b. $2\frac{3}{4}$ c. $1\frac{1}{8}$ d. $\frac{5}{16}$ e. $5\frac{5}{8}$
5. Perform the following additions in binary notation:

a. 11011	b. 1010.001	c. 11111	d. 111.11
+1100	+ 1.101	+ 0001	+ 00.01
<u> </u>	<u> </u>	<u> </u>	<u> </u>

3. a. $3\frac{1}{4}$ b. $5\frac{7}{8}$ c. $2\frac{1}{2}$ d. $6\frac{3}{8}$ e. $\frac{5}{8}$

1. Convert each of the following binary representations to its equivalent base ten form:
 - a. 101010 b. 100001 c. 10111 d. 0110 e. 11111
2. Convert each of the following base ten representations to its equivalent binary form:
 - a. 32 b. 64 c. 96 d. 15 e. 27
3. Convert each of the following binary representations to its equivalent base ten form:
 - a. 11.01 b. 101.111 c. 10.1 d. 110.011 e. 0.101
4. Express the following values in binary notation:
 - a. $4\frac{1}{2}$ b. $2\frac{3}{4}$ c. $1\frac{1}{8}$ d. $\frac{5}{16}$ e. $5\frac{5}{8}$
5. Perform the following additions in binary notation:

a. 11011	b. 1010.001	c. 11111	d. 111.11
+1100	+ 1.101	+ 0001	+ 00.01
<u> </u>	<u> </u>	<u> </u>	<u> </u>

4. a. 100.1 b. 10.11 c. 1.001 d. 0.0101 e. 101.101

1. Convert each of the following binary representations to its equivalent base ten form:

- a. 101010 b. 100001 c. 10111 d. 0110 e. 11111

2. Convert each of the following base ten representations to its equivalent binary form:

- a. 32 b. 64 c. 96 d. 15 e. 27

3. Convert each of the following binary representations to its equivalent base ten form:

- a. 11.01 b. 101.111 c. 10.1 d. 110.011 e. 0.101

4. Express the following values in binary notation:

- a. $4\frac{1}{2}$ b. $2\frac{3}{4}$ c. $1\frac{1}{8}$ d. $\frac{5}{16}$ e. $5\frac{5}{8}$

5. Perform the following additions in binary notation:

- | | | | |
|---------------|---------------|---------------|---------------|
| a. 11011 | b. 1010.001 | c. 11111 | d. 111.11 |
| +1100 | + 1.101 | + 0001 | + 00.01 |
| <u> </u> | <u> </u> | <u> </u> | <u> </u> |

5. a. 100111 b. 1011.110 c. 100000 d. 1000.00

Storing Integers

- **Two's complement notation:** The most popular means of representing integer values
- **Excess notation:** Another means of representing integer values
- Both can suffer from overflow errors.

Figure 1.21 Two's complement notation systems

a. Using patterns of length three

Bit pattern	Value represented
011	3
010	2
001	1
000	0
111	-1
110	-2
101	-3
100	-4

b. Using patterns of length four

Bit pattern	Value represented
0111	7
0110	6
0101	5
0100	4
0011	3
0010	2
0001	1
0000	0
1111	-1
1110	-2
1101	-3
1100	-4
1011	-5
1010	-6
1001	-7
1000	-8

Figure 1.22 Coding the value -6 in two's complement notation using four bits

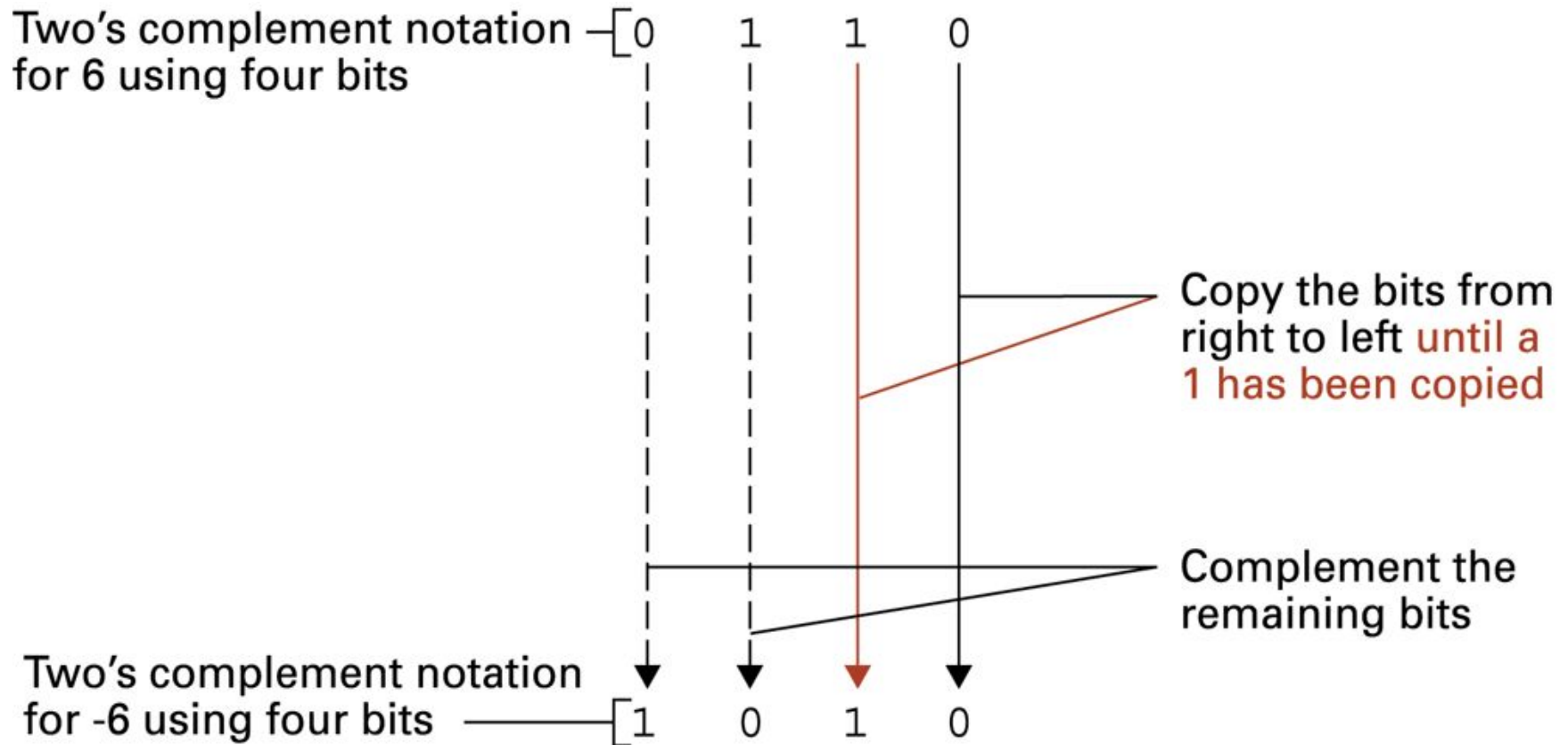


Figure 1.23 Addition problems converted to two's complement notation

Problem in base ten		Problem in two's complement		Answer in base ten
$\begin{array}{r} 3 \\ + 2 \\ \hline \end{array}$		$\begin{array}{r} 0011 \\ + 0010 \\ \hline 0101 \end{array}$		5
$\begin{array}{r} -3 \\ + -2 \\ \hline \end{array}$		$\begin{array}{r} 1101 \\ + 1110 \\ \hline 1011 \end{array}$		-5
$\begin{array}{r} 7 \\ + -5 \\ \hline \end{array}$		$\begin{array}{r} 0111 \\ + 1011 \\ \hline 0010 \end{array}$		2

Figure 1.24 An excess eight conversion table

Bit pattern	Value represented
1111	7
1110	6
1101	5
1100	4
1011	3
1010	2
1001	1
1000	0
0111	-1
0110	-2
0101	-3
0100	-4
0011	-5
0010	-6
0001	-7
0000	-8

Figure 1.25 An excess notation system using bit patterns of length three

Bit pattern	Value represented
111	3
110	2
101	1
100	0
011	-1
010	-2
001	-3
000	-4

1. Convert each of the following two's complement representations to its equivalent base ten form:
 - a. 00011 b. 01111 c. 11100
 - d. 11010 e. 00000 f. 10000
2. Convert each of the following base ten representations to its equivalent two's complement form using patterns of 8 bits:
 - a. 6 b. -6 c. -17
 - d. 13 e. -1 f. 0
3. Suppose the following bit patterns represent values stored in two's complement notation. Find the two's complement representation of the negative of each value:
 - a. 00000001 b. 01010101 c. 11111100
 - d. 11111110 e. 00000000 f. 01111111
4. Suppose a machine stores numbers in two's complement notation. What are the largest and smallest numbers that can be stored if the machine uses bit patterns of the following lengths?
 - a. four b. six c. eight

1. Convert each of the following two's complement representations to its equivalent base ten form:
- a. 00011 b. 01111 c. 11100
d. 11010 e. 00000 f. 10000
2. Convert each of the following base ten representations to its equivalent two's complement form using patterns of 8 bits:
- a. 6 b. -6 c. -17
d. 13 e. -1 f. 0
3. Suppose the following bit patterns represent values stored in two's complement notation. Find the two's complement representation of the negative of each value:
- a. 00000001 b. 01010101 c. 11111100
d. 11111110 e. 00000000 f. 01111111
4. Suppose a machine stores numbers in two's complement notation. What are the largest and smallest numbers that can be stored if the machine uses bit patterns of the following lengths?
- a. four b. six c. eight
1. a. 3 b. 15 c. -4 d. -6 e. 0 f. -16

1. Convert each of the following two's complement representations to its equivalent base ten form:
 - a. 00011 b. 01111 c. 11100
 - d. 11010 e. 00000 f. 10000
2. Convert each of the following base ten representations to its equivalent two's complement form using patterns of 8 bits:
 - a. 6 b. -6 c. -17
 - d. 13 e. -1 f. 0
3. Suppose the following bit patterns represent values stored in two's complement notation. Find the two's complement representation of the negative of each value:
 - a. 00000001 b. 01010101 c. 11111100
 - d. 11111110 e. 00000000 f. 01111111
4. Suppose a machine stores numbers in two's complement notation. What are the largest and smallest numbers that can be stored if the machine uses bit patterns of the following lengths?
 - a. four b. six c. eight

2.
 - a. 00000110 b. 11111010 c. 11101111
 - d. 00001101 e. 11111111 f. 00000000

1. Convert each of the following two's complement representations to its equivalent base ten form:
 - a. 00011 b. 01111 c. 11100
 - d. 11010 e. 00000 f. 10000
2. Convert each of the following base ten representations to its equivalent two's complement form using patterns of 8 bits:
 - a. 6 b. -6 c. -17
 - d. 13 e. -1 f. 0
3. Suppose the following bit patterns represent values stored in two's complement notation. Find the two's complement representation of the negative of each value:
 - a. 00000001 b. 01010101 c. 11111100
 - d. 11111110 e. 00000000 f. 01111111
4. Suppose a machine stores numbers in two's complement notation. What are the largest and smallest numbers that can be stored if the machine uses bit patterns of the following lengths?
 - a. four b. six c. eight

3. a. 11111111 b. 10101011 c. 00000100
d. 00000010 e. 00000000 f. 10000001

1. Convert each of the following two's complement representations to its equivalent base ten form:

a. 00011	b. 01111	c. 11100
d. 11010	e. 00000	f. 10000
2. Convert each of the following base ten representations to its equivalent two's complement form using patterns of 8 bits:

a. 6	b. -6	c. -17
d. 13	e. -1	f. 0
3. Suppose the following bit patterns represent values stored in two's complement notation. Find the two's complement representation of the negative of each value:

a. 00000001	b. 01010101	c. 11111100
d. 11111110	e. 00000000	f. 01111111
4. Suppose a machine stores numbers in two's complement notation. What are the largest and smallest numbers that can be stored if the machine uses bit patterns of the following lengths?

a. four	b. six	c. eight
---------	--------	----------

4.
 - a. With 4 bits the largest value is 7 and the smallest is -8. 1000 to 0111
 - b. With 6 bits the largest value is 31 and the smallest is -32.
 - c. With 8 bits the largest value is 127 and the smallest is -128. 1000 0000 to 0111 1111

5. In the following problems, each bit pattern represents a value stored in two's complement notation. Find the answer to each problem in two's complement notation by performing the addition process described in the text. Then check your work by translating the problem and your answer into base ten notation.

a.
$$\begin{array}{r} 0101 \\ + 0010 \\ \hline \end{array}$$

b.
$$\begin{array}{r} 0011 \\ + 0001 \\ \hline \end{array}$$

c.
$$\begin{array}{r} 0101 \\ + 1010 \\ \hline \end{array}$$

d.
$$\begin{array}{r} 1110 \\ + 0011 \\ \hline \end{array}$$

e.
$$\begin{array}{r} 1010 \\ + 1110 \\ \hline \end{array}$$

5. a. 0111 ($5 + 2 = 7$) b. 0100 ($3 + 1 = 4$) c. 1111 ($5 + (-6) = -1$)
 d. 0001 ($-2 + 3 = 1$) e. 1000 ($-6 + (-2) = -8$)

6. Solve each of the following problems in two's complement notation, but this time watch for overflow and indicate which answers are incorrect because of this phenomenon.

a.	0100	b.	0101	c.	1010	d.	1010	e.	0111
	+ 0011		+ 0110		+ 1010		+ 0111		+ 0001
	<u> </u>		<u> </u>		<u> </u>		<u> </u>		<u> </u>

7. Translate each of the following problems from base ten notation into two's complement notation using bit patterns of length four, then convert each problem to an equivalent addition problem (as a machine might do), and perform the addition. Check your answers by converting them back to base ten notation.

a.	6	b.	3	c.	4	d.	2	e.	1
	-(-1)		-2		-6		-(-4)		-5
	<u> </u>		<u> </u>		<u> </u>		<u> </u>		<u> </u>

6. Solve each of the following problems in two's complement notation, but this time watch for overflow and indicate which answers are incorrect because of this phenomenon.

a.	$\begin{array}{r} 0100 \\ + 0011 \\ \hline \end{array}$	b.	$\begin{array}{r} 0101 \\ + 0110 \\ \hline \end{array}$	c.	$\begin{array}{r} 1010 \\ + 1010 \\ \hline \end{array}$	d.	$\begin{array}{r} 1010 \\ + 0111 \\ \hline \end{array}$	e.	$\begin{array}{r} 0111 \\ + 0001 \\ \hline \end{array}$
----	---	----	---	----	---	----	---	----	---

7. Translate each of the following problems from base ten notation into two's complement notation using bit patterns of length four, then convert each problem to an equivalent addition problem (as a machine might do), and perform the addition. Check your answers by converting them back to base ten notation.

a.	$\begin{array}{r} 6 \\ -(-1) \\ \hline \end{array}$	b.	$\begin{array}{r} 3 \\ -2 \\ \hline \end{array}$	c.	$\begin{array}{r} 4 \\ -6 \\ \hline \end{array}$	d.	$\begin{array}{r} 2 \\ -(-4) \\ \hline \end{array}$	e.	$\begin{array}{r} 1 \\ -5 \\ \hline \end{array}$
----	---	----	--	----	--	----	---	----	--

- | | | |
|------------|--------------------|--------------------|
| 6. a. 0111 | b. 1011 (overflow) | c. 0100 (overflow) |
| d. 0001 | e. 1000 (overflow) | |

6. Solve each of the following problems in two's complement notation, but this time watch for overflow and indicate which answers are incorrect because of this phenomenon.

a.	$\begin{array}{r} 0100 \\ + 0011 \\ \hline \end{array}$	b.	$\begin{array}{r} 0101 \\ + 0110 \\ \hline \end{array}$	c.	$\begin{array}{r} 1010 \\ + 1010 \\ \hline \end{array}$	d.	$\begin{array}{r} 1010 \\ + 0111 \\ \hline \end{array}$	e.	$\begin{array}{r} 0111 \\ + 0001 \\ \hline \end{array}$
----	---	----	---	----	---	----	---	----	---

7. Translate each of the following problems from base ten notation into two's complement notation using bit patterns of length four, then convert each problem to an equivalent addition problem (as a machine might do), and perform the addition. Check your answers by converting them back to base ten notation.

a.	$\begin{array}{r} 6 \\ -(-1) \\ \hline \end{array}$	b.	$\begin{array}{r} 3 \\ -2 \\ \hline \end{array}$	c.	$\begin{array}{r} 4 \\ -6 \\ \hline \end{array}$	d.	$\begin{array}{r} 2 \\ -(-4) \\ \hline \end{array}$	e.	$\begin{array}{r} 1 \\ -5 \\ \hline \end{array}$
----	---	----	--	----	--	----	---	----	--

7. a.	$\begin{array}{r} 0110 \\ + 0001 \\ \hline 0111 \end{array}$	b.	$\begin{array}{r} 0011 \\ + 1110 \\ \hline 0001 \end{array}$	c.	$\begin{array}{r} 0100 \\ + 1010 \\ \hline 1110 \end{array}$	d.	$\begin{array}{r} 0010 \\ + 0100 \\ \hline 0110 \end{array}$	e.	$\begin{array}{r} 0001 \\ + 1011 \\ \hline 1100 \end{array}$
-------	--	----	--	----	--	----	--	----	--

Data Compression

- Lossy versus lossless
- Run-length encoding
- Frequency-dependent encoding
(Huffman codes)
- Relative encoding
- Dictionary encoding (Includes adaptive dictionary encoding such as LZW encoding.)

Compressing Images

- GIF: Good for cartoons
- JPEG: Good for photographs
- TIFF: Good for image archiving

Compressing Audio and Video

- MPEG
 - High definition television broadcast
 - Video conferencing
- MP3
 - Temporal masking
 - Frequency masking

Communication Errors

- Parity bits (even versus odd)
- Checkbytes
- Error correcting codes